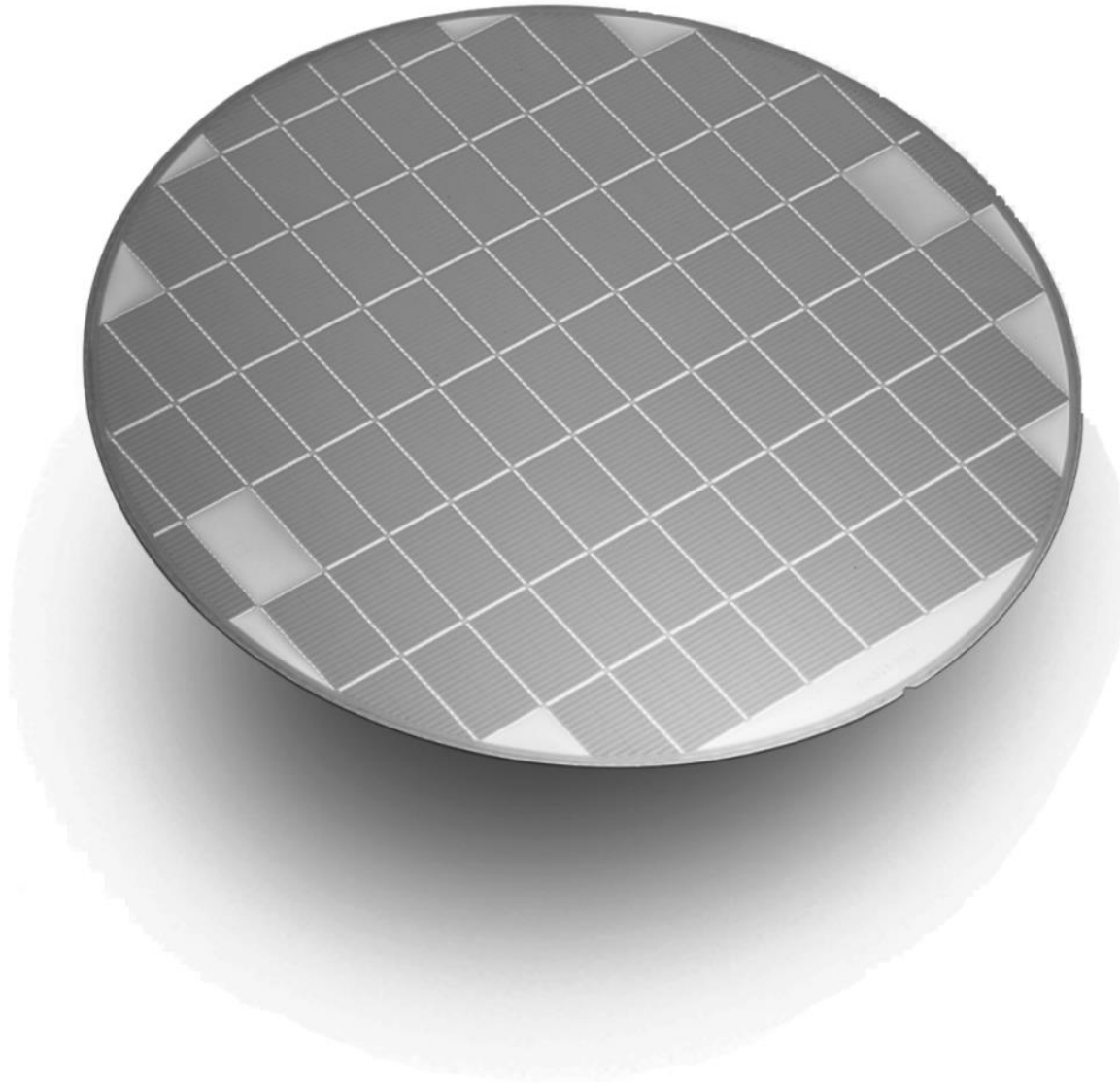


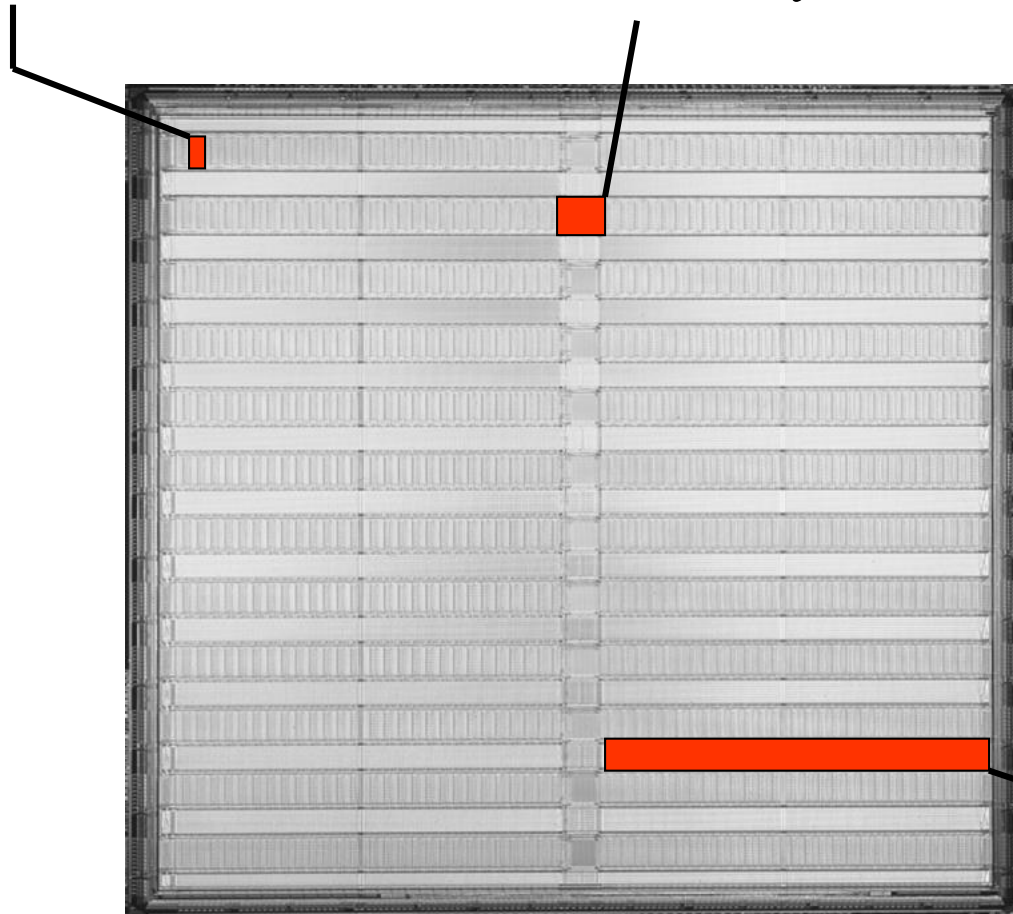
VHDL - Tutorial



FPGA - FIELD-PROGRAMMABLE GATE ARRAY

Group of 8 logic cells

Memory block



Interconnection
wires

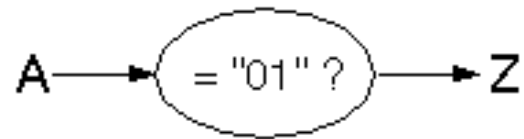
VHDL ?

- Very High Speed Integrated Circuit
Hardware Description Language
- American Department of Defense “DoD”

VHDL

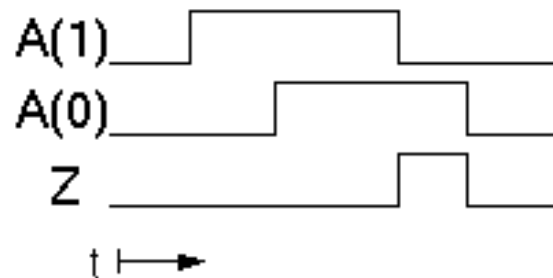
- Application of HDLs

Modelling

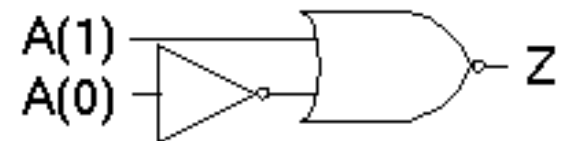


`Z <= '1' when A="01"
else '0' ;`

Simulation



Synthesis



VHDL - History

- **early `70s: Initial discussions**
- **late `70s: Definition of requirements**
- **mid -`82: Contract of development with IBM, Intermetrics and TI**
- **mid -`84: Version 7.2**
- **mid -`86: IEEE-Standard**
- **1987: DoD adopts the standard -> IEEE.1076**
- **mid -`88: Increasing support by CAE manufacturers**
- **late `91: Revision ➔ **Eu Comecei a Utilizar !****
- **1993: New standard**
- **1999: VHDL-AMS extension**

VHDL - History

Hardware Description

x

Software Description

General

- -----
-- Example VHDL Code --

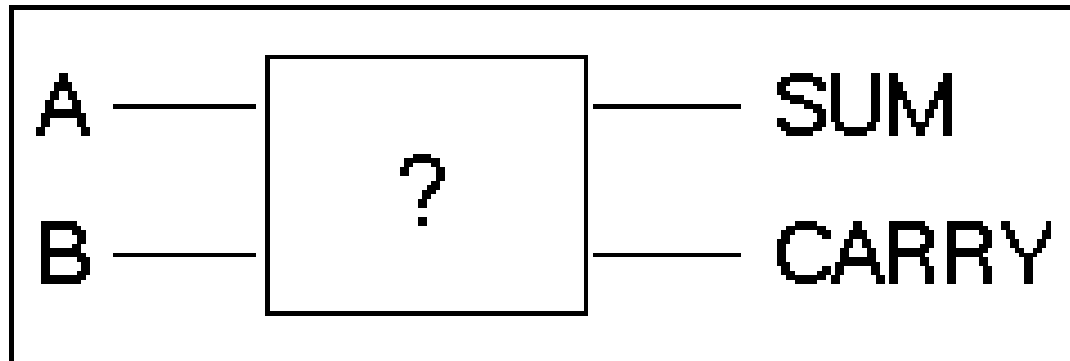
signal mySignal: bit; -- an example signal

MYsignal <= '0', -- start with '0'
 '1' AFTER 10 ns, -- and toggle after
 '0' after 10 ns, -- every 10 ns
 '1' afTer 10 ns;

- **Case insensitive**
- **Comments: '--' until end of line**
- **Statements are terminated by ';' (may span multiple lines)**
- **Signal assignment: '<='**
- **Variable assignment: ':='**
- **User defined names:**
 - letters, numbers, underscores
 - start with a letter

Entity

- **entity** HALFADDER is
 port(
 A, B: in bit;
 SUM, CARRY: out bit);
end HALFADDER;



Architecture

- entity HALFADDER is
 port(
 A, B: in bit;
 SUM, CARRY: out bit);
end HALFADDER;

architecture RTL of HALFADDER is

begin

SUM <= A xor B;
CARRY <= A and B;

end RTL;

Signal x Variable: Parallelism

- Signal:
 - Parallel:
 - $A \leq B;$
 - $B \leq A;$
- Variable:
 - Sequential:
 - $Aux := A;$
 - $A := B;$
 - $B := Aux;$

Variable Declaration

- `CONSTANT LOAD : STD_LOGIC_VECTOR (5 downto 0) := "110000";`
- `SIGNAL PC :STD_LOGIC_VECTOR(15 downto 0); -- Program Counter`
- `TYPE STATES IS (decode, exec, halted);`
- `SIGNAL STATE :STATES;`
- `TYPE REGISTERS IS ARRAY (0 to 15) OF STD_LOGIC_VECTOR(15 downto 0);`
- `SIGNAL REG :REGISTERS;`
- `VARIABLE RX : INTEGER;`
- `RX := "0001"; ou RX := conv_integer(IR1(9 DOWNT0 6));`
- `REG(RX) <= x"0000";`

Math

- $AUX := X + Y;$
- $AUX := X - Y;$
- $AUX := X * Y;$
- $AUX := CONV_STD_LOGIC_VECTOR(
CONV_INTEGER(X) / CONV_INTEGER(Y), 16
);$

State Machine

- PROCESS (clk, reset)
 - VARIABLE RX : INTEGER;
 - VARIABLE AUX :STD_LOGIC_VECTOR(15 downto 0);
- BEGIN
 - if (reset = '1') then
 - REG(0) <= x"0000";
 - State <= busca;
 - elsif (clk'event) and (clk = '1') then
 - CASE state IS
 - WHEN busca=>
 - » state <= decodifica;
 - WHEN decodifica=>
 - » state <= exec;
 - WHEN exec=>
 - » state <= busca;
 - WHEN OTHERS =>
 - » state <= busca;
 - END CASE;
 - end if;
 - END PROCESS;

State Machine

- CASE state IS -- Case das Transições
 - WHEN 0 =>
 - Transições
 - WHEN 1 =>
 - IF (FL = '1') THEN state <= 2; --Transicoes
 - ELSE state <= 1;
 - WHEN 2 =>
 - ...
 - WHEN OTHERS =>
 - state <= 0;
- END CASE;
- CASE state IS -- Case das Ações
 - WHEN 0 =>
 - Ações
 - WHEN 1 =>
 - buzina <= '1'; -- Acoes
 - WHEN 2 =>
 - ...
 - WHEN OTHERS =>
 - buzina <= '0'; -- Acoes
- END CASE;